

Unified Integration Architecture and Mainnet Deployment Plan

BANKON / pythai.net Ecosystem — DAIO, Parsec, x402, Kraken

(c) 2026 BANKON — all rights reserved. Apache-2.0. Python $\geq$ 3.12. Foundry canonical. PHP for payment servers. Flat snake_case (cypherpunk2048).

TL;DR

- **Mindx.pythai.net is the integration hub.** Agenticplace.pythai.net is the marketplace UI, bankon.pythai.net is the identity/payment primitive layer, and every cross-service call is metered through an x402 HTTP-402 gateway whose facilitator is GoPlausible's `x402-avm` (branded as Parsec) on Algorand mainnet (CAIP-2 `algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`) with an EVM rail on Base (USDC `0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`). The BONAFIDE nine-contract suite ships to **Polygon mainnet** through the **Agglayer Unified Bridge** at `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` (labeled "Polygon: Agglayer Unified Bridge" on Blockscan; "Polygon (Matic): zkEVM Bridge" on Etherscan) using the canonical `L1Escrow + ZkMinterBurner + NativeConverter` pattern from `BuildOnPolygon/usdc-lxly`, with constitutional state mirrored to Algorand via Wormhole NTT. [GitHub](#) [GitHub](#)
- **Parsec is wallet-agnostic.** It defines one signing/key-management contract — closely modelled on GoPlausible's `ClientAvmSigner` Protocol (`(address: str,` `sign_transactions(unsigned_txns, indexes_to_sign) -> list[bytes | None])`) generalised to multi-chain — and ships discrete backends as extensions: an Algorand state-proof / Falcon-aware backend, an ARC-52 BIP32-Ed25519 HD backend (Peikert mode, 8 safe derivation levels), a Pera native-multisig Shared-Accounts backend (LIVE since 21 April 2026, supporting "2-of-3, 3-of-5, weighted" topologies per the official @PeraAlgoWallet announcement), and the new **Kraken extension** which wraps both the open-source `krakenfx/wallet` React Native self-custody wallet (Bitcoin / Ethereum / Solana / Optimism / Base / Arbitrum / Polygon / Dogecoin / Blast) and the Kraken Pro REST + WebSocket v2 trading API as a single Parsec backend whose trade orders are gated by DAIO Boardroom / War Council verdicts. [GitHub](#) [Genfinity](#)
- **DAIO governance is real and binding.** The thirteen Sun Tzu assessor agents emit verdicts that aggregate into one of four actions — **WAGE** ($\geq 10/13$), **SUBDUE** ($\geq 8/13$), **HOLD** ($5-7/13$), **WITHDRAW** ($< 5/13$) — and that aggregation authorises any Kraken `AddOrder`, any Parsec wallet signature over treasury funds, and any cross-chain message through the openBDK / Wormhole bridge. The deployment is mainnet, the test harness is Foundry, and the falsification suite is Popperian: every claim a contract makes about itself is paired with a test designed to break it.

Key Findings

The user's ecosystem has converged onto a small set of strong primary-source primitives that materially constrain the architecture, and the design below is structured around those constraints rather than imagined ones.

First, **GoPlausible's x402-avm is the canonical Algorand x402 implementation**, distributed both as the scoped npm family (`@x402-avm/core`, `@x402-avm/avm`, `@x402-avm/express`, `@x402-avm/next`, `@x402-avm/extensions`) and as the Python package (`x402-avm`) (pip-installable with `[avm]`, `[fastapi]`, `[flask]`, `[extensions]` extras). It uses **CAIP-2 network identifiers** in V2, where Algorand mainnet is the literal string `algorand:wGHE2Pwddvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=` (the base64 mainnet genesis hash) and Algorand testnet is `algorand:SG01GKSzyE7IEPItTxCBYw9x8FmnrCDexi9/c0UJ0iI=`; V1 string identifiers (`algorand-mainnet`, `algorand-testnet`) are kept for backward compatibility through automatic mapping. Per GoPlausible's own changelog, **v2 → v2.6+ removed `algosdk` as a dependency in favour of `@algorandfoundation/algokit-utils@10.0.0-alpha.39`**, which means any Parsec wrapper must depend on AlgoKit Utils, not `algosdk` directly. The mechanism the user must consume is `ExactAvmScheme`, registered on the client with `register_exact_avm_client(client, signer)` and on the server with `registerExactAvmScheme(server)`; the AVM "exact" scheme constructs a two-transaction atomic group (the buyer's ASA transfer plus a fee-payer transaction signed by the facilitator), allowing **gasless payments** because the resource server's facilitator pays Algorand's tiny fees. USDC ASA `31566704` on mainnet (decimals 6, manager `37XL...HAL4`, reserve `Centre`) is the canonical asset. [GitHub + 7](#)

Second, **Coinbase's x402 specification on EVM uses EIP-3009 `transferWithAuthorization`** with a fallback `Permit2` + `x402ExactPermit2Proxy` "witness pattern" for non-EIP-3009 tokens, and an ERC-7710 path for smart-account delegated payments. The headers are V2-style: `PAYMENT-REQUIRED` (server → client, base64-encoded JSON of accepted `PaymentRequirements`), `PAYMENT-SIGNATURE` (client → server retry, base64-encoded signed authorization), `PAYMENT-RESPONSE` (server → client, success status with transaction hash). x402 charges no protocol fee, and per the Coinbase Developer Docs (docs.cdp.coinbase.com/x402/welcome) the CDP facilitator provides "a generous free tier (1,000 transactions/month, then \$0.001/transaction)" covering "All ERC-20 payments on Base, Polygon, Arbitrum, World, and Solana — via EIP-3009 (USDC, EURC) or Permit2 (any ERC-20)." The intent to create the x402 Foundation was announced by Coinbase + Cloudflare on 23 September 2025, and the Foundation was formally launched under the Linux Foundation on 2 April 2026 at the MCP Dev Summit North America in New York, with 22 founding members including Adyen, AWS, American Express, Circle, Google, Mastercard, Microsoft, Polygon Labs, Shopify, Solana

Foundation, Stripe, and Visa (Linux Foundation press release, 2 April 2026). Per Sherlock.xyz ("x402 Explained"): "As of March 2026, x402 has processed over 119 million transactions on Base and 35 million on Solana, handles roughly \$600 million in annualized volume." Stripe added x402 support on 10 February 2026 via PaymentIntents using USDC on Base (announced by Stripe product manager Jeff Weinstein); Google's Agent Payments Protocol (A2A) incorporates x402 as its agent-to-agent crypto settlement path. (GitHub + 11)

Third, **Kraken Wallet is the React Native repo** at (github.com/krakenfx/wallet) (TypeScript, MIT licensed per the GitHub org page), with a Realm-backed local store, an Electrum server proxy via (wallet.kraken.com/api/data) (Harmony backend) and (wallet.kraken.com/api/push) (GroundControl), and WalletConnect for dApp integration. iOS signing happens in the sandboxed app environment because Apple's CryptoKit does not support secp256k1; this is a known security trade-off Kraken called out at launch. The wallet supports Bitcoin, Ethereum mainnet, Solana, Optimism, Base, Arbitrum, Polygon, Dogecoin and Blast. The **Kraken Pro API** uses **HMAC-SHA512(URI path + SHA256(nonce + POST data))** with the base64-decoded secret as the HMAC key; nonces are always-increasing 64-bit integers (typically `int(time.time()*1000)`) and Kraken supports a per-key nonce window for tolerating out-of-order delivery. The current REST endpoint set is `/0/private/{AddOrder, CancelOrder, QueryOrders, OpenOrders, ClosedOrders, Balance, TradeBalance, GetWebSocketsToken, ...}` and the public set is `/0/public/{Ticker, OHLC, Depth, Trades, ...}`. **WebSocket v2** at `wss://ws.kraken.com/v2` and `wss://ws-auth.kraken.com/v2` consolidates v1's `openOrders` + `ownTrades` into a single `executions` channel ("the executions channel sends both order updates, corresponding to the 'openOrders' channel in WS API v1, and trade execution updates, corresponding to the 'ownTrades' channel in WS API v1") and uses RFC3339 timestamps and ISO-style pair symbols (`BTC/USD`, not `XBT/USD`). Authentication tokens for private feeds are obtained from REST `GetWebSocketsToken`. Per Kraken's change log, the Futures API authentication is changing on **1 October 2025** to hash encoded query parameters; deployment scripts must pin to the new Auth generation. (GitHub + 2)

Fourth, the dual-chain deployment topology is well-defined by ([BuildOnPolygon/usdc-lxly](https://github.com/BuildOnPolygon/usdc-lxly)) as the canonical pattern. Three contracts manage minting/burning across L1/L2 — **L1Escrow** on Ethereum L1 (or any L1 of choice) receives the L1 asset and messages the L2 mint contract through the Polygon Unified Bridge; **ZkMinterBurner** on L2 receives the message and mints/burns USDC-e (in our case, BonaToken); **NativeConverter** on L2 lets users swap bridge-wrapped tokens for native ones and exposes a permissionless `migrate()` to return wrapped tokens to L1 escrow. The Unified Bridge address `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` is verified-canonical on both L1 (Etherscan label "Polygon (Matic): zkEVM Bridge") and Polygon zkEVM L2 (zkevm.polygonscan.com label "Polygon Hermez: Bridge"), and Blockscan labels it explicitly "Polygon: Agglayer Unified Bridge." Per Polygon's own docs, "the unified bridge (prev. LxLy bridge) is the common access point for chains connected to the Agglayer ... a shared smart contract deployed on Ethereum, responsible for interoperability between chains in the form of cross-chain transaction and L2 to L2 transfers."

The MakerDAO/Sky **Chainlog** at `(0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F)` (the `(sky-ecosystem/dss-chain-log)` README defines it as "An auth-managed on-chain changelog for the MCD system") is reused here as a registry pattern for BONAFIDE — the Aerarium contract is the project's own chainlog, exposing `(list())`, `(getAddress(bytes32 key))`, `(count())`, `(get(uint256 i))`, `(version())`, `(wards(address))`. `(GitHub + 4)`

Fifth, the **"1 Relayer + 3 Validator BFT" topology in the user-supplied spec is not a canonical Agglayer pattern**. Agglayer's actual settlement model is ZK-proof-based (Unified Bridge + Pessimistic Proof + State Transition Proof), and the closest validator-style construct, Polygon's Stake-the-Bridge (STB), uses three named roles — Admin, Escrow manager, Risk manager — not a BFT quorum. The deployment plan below therefore uses a different but equivalent trust-minimised topology: a single **Relayer** service that observes Algorand state-proof emissions and EVM event logs, plus three **Validator co-signers** that form a Gnosis Safe 2-of-3 over the BONAFIDE Senatus contract (the role the user labelled "Validator BFT"), and the Pessimistic Proof for any chain that opts into Agglayer settlement. This re-mapping preserves the user's design intent (trustless cross-chain witness with redundant verifiers) while aligning to named, audited, in-production primitives. A Wormhole NTT track is also kept for Algorand because Wormhole NTT is the in-production Algorand bridge ("NTT ... enables direct token transfers by either burning tokens on the source chain and minting them on the destination, or by locking tokens on one chain while minting representations on others"), with the Wormhole Algorand core, token bridge, and stateless VAA-verify program already deployed. `(Polygon + 2)`

Sixth, **Algorand state proofs use Falcon-512 signatures** as the post-quantum signing scheme over compact certificates of the last 256 block headers, and on **3 November 2025 the Algorand Foundation executed the first post-quantum transaction on live mainnet using Falcon-1024 signatures** ("Algorand Post-Quantum Ledger", algorand.co/blog: "It builds on Algorand's November 2025 post-quantum milestone: the first MainNet transaction authorized with Falcon signatures, implemented through an account abstraction"), so the Parsec quantum-awareness extension is not speculative but a wrapper over a production primitive. **ARC-52** is implemented across TypeScript (`(@algorandfoundation/xhd-wallet-api)`), Kotlin, Swift, and Go libraries, all derived from BIP32-Ed25519 over a non-linear keyspace, with **Peikert mode as the default (8 safe levels of derivation)** and Khovratovich available for legacy interop; the cointype is **283' for Algorand addresses and 0' for identity keys**, with derivation path `(m'/44'/283'/account'/change/keyIndex)`. **Pera Wallet Shared Accounts went live on 21 April 2026** with native protocol-level multisig (no smart contract overhead) following ARC-35 wallet backup conventions and supporting "2-of-3, 3-of-5, weighted" topologies (per the official @PeraAlgoWallet announcement), providing a clean wrapping target for the Parsec Pera-multisig extension. `(BitcoinEthereumNews.com...)`

Seventh, the user's existing identity layer (BANKON Wallet) is on **Polygon via Alchemy Account Kit smart-account authentication** (Google/Apple/email backends) and is intended to integrate with **RainbowKit**. This means BONAFIDE's identity entry point — `(Fides)` — is itself a

smart-account whose owner key is rotatable via Algorand-style rekeying on the EVM side using ERC-7710 delegation, mirrored by Algorand's native rekey on the AVM side.

Details

1. The three-service topology and the BONAFIDE attestation handshake

The three-service ecosystem is wired so that **every state-changing call enters through mindx.pythai.net**. Agenticplace.pythai.net is the marketplace front-end and serves the canonical chain registry at `agenticplace.pythai.net/allchain.html` (described below); it never touches private keys and never calls bankon.pythai.net directly. Bankon.pythai.net is the identity and payment primitive: it owns the BANKON Wallet smart-account onboarding flow, holds the Parsec wallet extension registry, and exposes a single attestation endpoint that proves "this account passed BONAFIDE Fides verification at time T." Mindx is the orchestration brain: it takes a user intent (from agenticplace) or an autonomous agent goal (from automindx / MASTERMIND), routes it through the DAIO Boardroom for verdict aggregation, signs whatever transactions are required via Parsec, meters every internal hop with x402, and writes the resulting state into the Tabularium (the BONAFIDE ledger contract).

The **BONAFIDE attestation handshake** runs on every cross-service request. A client (human or agent) requests an mindx endpoint with no headers; mindx returns HTTP 402 with a `PAYMENT-REQUIRED` envelope listing two `accepted` payment options — one for Algorand mainnet (`algorand:wGHE2Pwdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`), USDC ASA (`31566704`) and one for Base (`eip155:8453`), USDC (`0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913`). If the client is itself an agent calling another agent, the call is metered at fractions of a cent (typical `$0.001`–`$0.01`) and uses the AVM rail; if the client is a human checkout flow from agenticplace, the call is metered at the marketplace listing price and may use either rail. The retry carries a `PAYMENT-SIGNATURE` header containing the signed authorization, and an additional `X-BONAFIDE-FIDES` header carrying a JWT-style attestation signed by bankon.pythai.net's Fides oracle key. Mindx verifies the payment via the Parsec facilitator (which wraps GoPlausible's facilitator on AVM and a local EIP-3009 facilitator on Base), verifies the Fides attestation against bankon's public key, settles, and returns the resource with `PAYMENT-RESPONSE` containing the on-chain transaction hash. The attestation is namespaced exactly as in the user's RAGE doc — `{ "v": 1, "domain": { "app": "BONAFIDE", "origin": "bankon.pythai.net", "env": "prod", "action": "attest", "subject": { "address": "...", "kyc_level": "N", "content_hash": "sha256:...", "nonce": "...", "ts": "...", "exp": "..." } }, ... }` with canonical-JSON signing — and the same envelope is used by RAGE for capsule promotion, which makes the namespace genuinely unified. (GitHub)

The internal API contracts are FastAPI-served on mindx, declared with the **GoPlausible Bazaar discovery extension** so any facilitator can index the endpoints by capability. Each route declares

its input schema, output schema, and example via `declare_discovery_extension(...)` and registers `bazaarResourceServerExtension` so the runtime HTTP method is recorded automatically. This gives the marketplace a free machine-readable catalogue and makes agent-to-agent discovery a property of the protocol rather than out-of-band documentation. [GitHub](#)

2. The `allchain.html` CAIP-2 chain registry

`agenticplace.pythai.net/allchain.html` is treated as the canonical CAIP-2 chain manifest. The integration loads it on mindx startup, parses it into an in-memory `ChainRegistry`, and exposes the result through a single function `resolve_chain(caip2: str) -> ChainSpec`, where `ChainSpec` carries the RPC endpoint, the bridge contract (if any), the native gas token symbol, the canonical stablecoin asset id, the explorer URL, and the Parsec wallet extension capable of signing for that chain. Algorand mainnet resolves to the genesis-hash-base CAIP-2 string `algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`; the user's **Arc Testnet chainId 5042002** resolves to `eip155:5042002`; Polygon PoS to `eip155:137`; Polygon zkEVM to `eip155:1101`; Base mainnet to `eip155:8453`; Ethereum to `eip155:1`. New chains are added by editing `allchain.html` with a pull request against the agenticplace repo; mindx reloads the manifest on a signed webhook from a CI step that publishes the new file. Routing logic is mechanical — given a target chain and an asset, the orchestrator selects the cheapest rail by reading a per-chain median-gas oracle (Algorand: 0.001 ALGO; Base: ~\$0.0001; Polygon: ~\$0.001) and the per-chain x402 facilitator success rate over the past 24h. CAIP-2 in V2 of x402-avm makes the same payment envelope describe any chain, so the routing decision is a one-line dictionary lookup rather than per-chain code. [Eco](#)

3. Parsec wallet — extension interface specification

Parsec is the wallet-agnostic abstraction layer. Its core is a single Python Protocol (mirrored in TypeScript) named `ParsecWallet`, structurally typed so any backend that satisfies the methods qualifies as an extension. The contract is intentionally narrow:

```
python
```

```

class ParsecWallet(Protocol):
    extension_id: str # "kraken", "pera-multisig", "arc52-hd", "algorand-
    supported_chains: list[str] # CAIP-2 identifiers from allchain.html
    def addresses(self, chain: str) -> list[str]: ...
    def sign_transactions(
        self,
        chain: str,
        unsigned_txns: list[bytes],
        indexes_to_sign: list[int],
    ) -> list[bytes | None]: ...
    def capabilities(self) -> set[str]: # {"trade", "state_proof", "hd_derive", "mu
        ...

```

This is deliberately a near-verbatim generalisation of GoPlausible's `ClientAvmSigner` Protocol so that the Algorand state-proof and ARC-52 backends are direct conformers; for EVM backends, the bytes returned are RLP-encoded signed transactions or EIP-712 typed-data signatures depending on `capabilities`. A trading-capable extension additionally implements `place_order`, `cancel_order`, `open_orders`, `balance`, and `subscribe_executions` on a coroutine interface; non-trading extensions omit these and simply do not advertise `"trade"` in their capability set.

The shipped extensions are: (a) **algorand-falcon** — wraps the production Falcon-1024 logic signature scheme for post-quantum accounts, talks to algod via AlgoKit Utils; (b) **arc52-hd** — wraps `@algorandfoundation/xhd-wallet-api` for BIP32-Ed25519 derivation `m'/44'/283'/a'/c/k` in Peikert mode, both `KeyContext.Address` and `KeyContext.Identity`; (c) **pera-multisig** — wraps Pera Wallet Shared Accounts native multisig where a 2-of-3 (or n-of-m or weighted) approval gate is enforced by Algorand protocol itself, no smart-contract overhead, no gas premium; (d) **kraken** — the new extension covering both Kraken Wallet self-custody and Kraken Pro trading (next section); (e) reference EVM extensions for Base, Polygon, and Ethereum that wrap an HD seed via `viem`/`ethers` with optional Ledger and WalletConnect transports. The Parsec orchestrator picks the extension by chain capability lookup and never reaches into key material directly; private keys live behind the extension boundary and the only thing crossing the boundary is signed bytes. `Genfinity`

4. The Kraken extension — both wallet and trading API

The Kraken extension lives in `parsec_extensions/kraken/` as a flat snake_case module. Its custody half wraps `github.com/krakenfx/wallet` (TypeScript / React Native, MIT licensed) by speaking to a local RPC bridge — for production we run a thin Node service on `bankon.pythai.net` that exposes the wallet's Realm-backed key store over a Unix domain socket, with the Algorand state-proof verification and the Electrum/Harmony backends proxied through `wallet.kraken.com/api/data` exactly as in the open-source build. The supported

chains (Bitcoin, Ethereum, Solana, Optimism, Base, Arbitrum, Polygon, Dogecoin, Blast) cover everything the user actually needs on the EVM/UTXO side, and the wallet's WalletConnect integration lets it participate as a session on any dApp the agenticplace marketplace lists.

The trading half is a pure HTTP/WebSocket client. REST authentication is the canonical Kraken HMAC-SHA512: the API-Sign header is the base64 of `HMAC_SHA512(base64_decode(secret), URI_path + SHA256(nonce + urlencoded_post_data))`. The nonce is `int(time.time()*1000)` (milliseconds since epoch) and is required to be strictly increasing; the implementation uses a single Redis-backed atomic counter so multiple worker processes sharing one API key cannot interleave nonces, and the per-key nonce window is configured generously in the Kraken account UI to absorb network jitter. The endpoint set the trading agent uses in production is `POST /0/private/AddOrder` (placing limit/market/iceberg/stop-loss orders, with `pair`, `type`, `ordertype`, `price`, `volume`, optional `cl_ord_id` for idempotency, optional `oflags` for fcib/post/no_mpp deprecation note), `POST /0/private/CancelOrder` and `CancelOrderBatch`, `POST /0/private/OpenOrders` and `ClosedOrders`, `POST /0/private/QueryOrders`, `POST /0/private/Balance` and `TradeBalance`. Market data comes from `GET /0/public/{Ticker,OHLC,Depth,Trades}`. Rate limits are enforced by tier (Starter / Intermediate / Pro) and the AddOrder/CancelOrder counter is separate from the general counter; the implementation respects the documented increment and decay rates and degrades gracefully under 429 by exponential backoff. WebSocket subscription uses v2 at `wss://ws-auth.kraken.com/v2` after fetching the auth token from REST `/0/private/GetWebSocketsToken`; the consolidated `executions` channel replaces v1's `openOrders` + `ownTrades` and is the single source of truth for both order state transitions and fills. Reconnect logic respects Cloudflare's ~150 attempts/10-minute IP ban threshold by attempting instant reconnection only a small handful of times and then backing off to no faster than once every five seconds. (Kraken)

The DAIO gate sits between the Parsec call and the Kraken extension: every `place_order` request the extension receives goes through `daio_authorise(intent)` first, which submits the proposed trade to the Boardroom and the War Council and waits for the verdict. WAGE ($\geq 10/13$ assessors approve) means execute at full size, SUBDUE ($\geq 8/13$) means execute at 50% size with a tighter stop, HOLD (5–7/13) means defer to the next 5-minute aggregation window, and WITHDRAW ($< 5/13$) means cancel the intent and emit an on-chain veto log via the Censura contract. The thirteen assessors map one-to-one to the thirteen chapters of *The Art of War* — Laying Plans, Waging War, Attack by Stratagem, Tactical Dispositions, Energy, Weak Points and Strong, Manoeuvring, Variation in Tactics, The Army on the March, Terrain, Nine Situations, The Attack by Fire, The Use of Spies — each a small LLM-prompted scoring agent run inside automindx with deterministic temperature 0 and a published prompt hash. The Tabularium ledger records the full assessor vector for every authorised trade, which makes the system auditable end-to-end.

5. x402 metering implementation

The metering implementation is one module, `mindx_x402/`, served from FastAPI. Every `mindx` endpoint declares its `RouteConfig` exactly as in GoPlausible's example:

```
python

from x402.http.types import RouteConfig
from x402.http import PaymentOption
from x402.extensions.bazaar import declare_discovery_extension

routes = {
    "POST /agent/{id}/invoke": RouteConfig(
        accepts=[
            PaymentOption(scheme="exact", pay_to=BONAFIDE_ALGORAND_ADDR,
                           price="$0.001",
                           network="algorand:wGHE2Pwvdvd7S12BL5FaOP20EGYesN73ktiC1qzkkit8="),
            PaymentOption(scheme="exact", pay_to=BONAFIDE_BASE_ADDR,
                           price="$0.001", network="eip155:8453"),
        ],
        description="Invoke agent and stream response",
        mime_type="application/x-ndjson",
        extensions={**declare_discovery_extension(input=..., output=...)}),
    },
}
```

The facilitator client is

`HTTPFacilitatorClient(url="https://parsec.pythai.net/facilitator")`. The facilitator itself, branded Parsec, is a thin Apache-2.0 PHP server that forwards AVM payments to GoPlausible's facilitator (or runs the GoPlausible facilitator-signer in-process when self-hosting) and handles EVM payments locally via an EIP-3009-aware signer keyed by a Polygon mainnet hot wallet topped up from the Aerarium treasury. Because the AVM "exact" scheme builds a two-transaction atomic group with a fee-payer, the buyer never needs ALGO — the facilitator's signer pays the round fee, the buyer pays only the USDC ASA transfer. This is the key UX property: an end user with only USDC ASA can transact instantly, which is exactly the property an agentic marketplace requires.

The PHP server reference is short by design; the user already shipped a PHP x402 library, so the Parsec wrapper re-exports its public functions and adds a

`parsec_facilitate($payment_header, $route_config)` entry that branches on `network`: for `algorand:*` it calls GoPlausible's Python facilitator over a local Unix socket (via `proc_open` to a long-lived Python worker), for `eip155:*` it calls the user's existing PHP EIP-3009 settler. The whole server is ~250 lines.

6. The DAIO mainnet deployment — Foundry, openBDK, Wormhole

The BONAFIDE nine-contract suite — **Genius** (the AI brain registry), **BonaToken** (the ERC-20 governance/utility token, EIP-3009 enabled for native x402 support), **Tabularium** (the audit ledger), **Fides** (identity attestation), **SponsioPactum** (programmable promise/escrow), **Censura** (veto and recall), **Senatus** (Gnosis Safe 2-of-3 over Validator co-signers), **Tessera** (membership tokens for Boardroom seats), plus **Aerarium** (treasury with Chainlog-style registry, modelled on MakerDAO's `dss-chain-log` at `0xdA0Ab1e0017DEbCd72Be8599041a2aa3bA7e740F`) and **Curia** (the executor / scheduler) — deploys to Polygon PoS mainnet under a single Foundry workspace.

The Foundry layout is flat snake_case: `src/genius.sol`, `src/bona_token.sol`, `src/tabularium.sol`, `src/fides.sol`, `src/sponsio_pactum.sol`, `src/censura.sol`, `src/senatus.sol`, `src/tessera.sol`, `src/aerarium.sol`, `src/curia.sol`, plus `src/openbdk/l1_escrow.sol`, `src/openbdk/zk_minter_burner.sol`, `src/openbdk/native_converter.sol` adapting the `BuildOnPolygon/usdc-lxly` pattern verbatim to BonaToken with the Unified Bridge address `0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` hard-coded as an immutable in the constructors. The mainnet go-live runbook proceeds in eight tested phases — fork-test on a local anvil instance (`anvil --fork-url $POLYGON_RPC --chain-id 137`), deploy the Senatus Gnosis Safe (2-of-3 with three Validator co-signers), deploy Aerarium and seed its Chainlog with the eventual contract addresses as `bytes32` keys, deploy BonaToken with EIP-3009 support and grant mint authority to ZkMinterBurner only, deploy Tabularium / Fides / SponsioPactum / Censura / Tessera / Genius / Curia in dependency order, wire the openBDK L1Escrow on Ethereum mainnet and ZkMinterBurner on Polygon zkEVM (or Polygon PoS) with NativeConverter, deploy the Wormhole NTT manager and Wormhole Transceiver on both Algorand and the chosen EVM chain following the Algorand Foundation's NTT deployment guide, run the full Foundry test suite against the deployed addresses with `forge test --fork-url $RPC -vvv`, and finally cut the Aerarium owner from the deployer EOA to the Senatus Safe and renounce the deployer. [GitHub](#)

Foundry tests follow Popperian falsification: every property the contracts claim is paired with at least one test that tries to break it. `test_BonaToken_RevertWhen_NotMinter_Mints` uses `vm.prank(address(0xBAD))` plus `vm.expectRevert(Unauthorized.selector)` and calls `bona.mint(victim, 1e18)`; `test_Senatus_RevertWhen_OneOfTwoSign` simulates only one Safe owner signing and expects the multisig threshold check to revert; `test_Curia_VerdictWage_ExecutesAtFullSize`, `test_Curia_VerdictSubdue_ExecutesAtHalf`, `test_Curia_VerdictHold_DefersToNextWindow`, and `test_Curia_VerdictWithdraw_EmitsCensuraVeto` cover the verdict ladder with `vm.expectEmit(true, true, false, true)` against the expected `Verdict` and `Veto` events; the openBDK round-trip is tested by deploying both L1 and L2 fork instances, calling `l1_escrow.bridgeToken(amount, recipient)`, simulating the Unified Bridge `claimMessage`,

and asserting that `usdc_e.balanceOf(recipient) == amount` then reversing the flow with `zk_minter_burner.bridgeToken(amount, recipient)` and asserting the L1 balance restored. Property tests via `forge fuzz` exercise BonaToken's EIP-3009 path with random `validAfter`/`validBefore`/`nonce` triples and assert that no signature can be replayed across nonces. (Foundry) (Foundry)

The deployment script `script/deploy_mainnet.s.sol` uses Forge's `vm.startBroadcast(vm.envUint("DEPLOYER_PK"))` and after every contract creation writes the address into a JSON output via `vm.writeJson(addr.toHexString(), "deployments/mainnet.json", key)`. The post-deployment verification step runs `forge verify-contract --chain polygon --etherscan-api-key $POLYGONSCAN_KEY` for every contract using flat-file output from `forge flatten`, then runs `cast call` against each contract to assert its `version()`, its `owner()` (must be the Senatus Safe), and its `wards()` table. Rollback is a Senatus-signed Aerarium chainlog entry that re-points a logical key (say `BONAFIDE_TABULARIUM`) to a previously-deployed alternate address — because Aerarium is the source-of-truth registry and every other contract resolves dependencies through it on each call, swapping a key swaps the live implementation atomically without redeploying the consumers.

The Algorand half of the dual-chain is the constitutional layer. **BonaToken** is mirrored as an ASA created with `manager=Senatus-algo-msig`, `reserve=Aerarium-algo`, `freeze=Censura-algo`, decimals 6, total supply matching the EVM cap. The constitutional contracts (Fides identity attestations and Tabularium audit logs) live as Algorand stateful smart contracts written in `algopy` / PyTEAL, deployed via AlgoKit, and bridged to the EVM via Wormhole NTT in **locking mode on Algorand** (Algorand is the canonical home of the ASA; EVM mints proportionally when Algorand locks). The Wormhole Transceiver verifies VAAs from a supermajority of Guardians, and Algorand state proofs (Falcon-signed, every 256 rounds) anchor the historical record so external chains can verify Algorand consensus decisions in a post-quantum-safe manner. This is the layer that makes the "constitutional vs economic" split real: the Algorand side is slow-changing and quantum-resistant for identity and history; the EVM side is fast-changing and DeFi-composable for economic activity. (Circle + 3)

The user's spec mentioned a "1 Relayer + 3 Validator BFT topology." Because this is not a canonical Agglayer pattern (Agglayer is ZK-proof-based and Polygon's Stake-the-Bridge instead names three roles — Admin / Escrow manager / Risk manager — rather than a BFT quorum), the deployment uses a substitute that preserves the intent: one **Relayer** service (a single hardened EC2 instance running an event listener on both chains) plus three **Validator co-signers** wired as the Senatus 2-of-3 Gnosis Safe on Polygon and as a 2-of-3 Pera-style native multisig on Algorand. This delivers redundancy and trust minimisation without inventing a non-existent canonical pattern. If/when Agglayer Pessimistic Proofs become available for BonaToken's chain, the L1Escrow flows are upgraded to consume them and the Relayer becomes a thin proof-submission cron.

7. Innerstanding and operational closure

The system closes as follows. A user lands on `agenticplace.pythai.net`, picks an agent, the marketplace UI POSTs to `mindx` with no payment header, `mindx` returns 402 with Algorand + Base payment options, the user's Parsec wallet (Kraken extension by default, or Pera multisig, or ARC-52 HD, or Algorand-Falcon for quantum users) signs an `x402-avm` exact-scheme atomic group whose fee-payer is the Parsec facilitator. `mindx` receives the signed payload, the facilitator verifies and settles on Algorand (~3 seconds, fractions of a cent in fees paid by the facilitator from the Aerarium hot wallet), `mindx` emits a Tabularium log of the access, and routes the request to `automindx` which loads the chosen agent. If the agent is a trading agent, every order flows through `daio_authorise` → 13-assessor Boardroom verdict → Curia executor → Parsec Kraken extension → Kraken Pro REST `AddOrder`. Order acks and fills stream back via the WebSocket v2 `executions` channel, are logged into Tabularium, and feedback updates the assessor priors. Treasury rebalancing across Polygon, Base, Algorand and Kraken custody is itself a DAIO-gated action that signs through the Senatus Safe on EVM and the Pera Shared Account on AVM. The user — Gregory / Professor Codephreak — observes the whole loop through a NiceGUI dashboard that exposes the Tabularium feed, the open-orders WS stream, and the Boardroom verdicts in real time.

Recommendations

Deploy in three staged phases keyed to concrete on-chain and off-chain benchmarks.

Phase one (week 0-2): facilitator and Parsec extension shell. Stand up Parsec as a Python 3.12 package with the protocol-level `ParsecWallet` interface and three extensions — `arc52-hd`, `pera-multisig`, `kraken` — implementing only the read-only methods (`addresses`, `capabilities`, `balance` for Kraken). Stand up the PHP Parsec facilitator on `bankon.pythai.net`, registered against GoPlausible's hosted facilitator for AVM and against a self-hosted EIP-3009 settler on Base. Verify by running Bazaar discovery against a single `/health` endpoint priced at `$0.001` and watching a Pera Wallet sign the atomic group end-to-end. **Benchmark to pass:** ten consecutive AVM and EVM x402 settlements with median latency under three seconds and zero invalid nonce errors. **Stop condition:** if Kraken Wallet's secp256k1 signing path inside the React Native sandbox shows any memory leak or key exposure under fuzzing, fall back to making the Kraken extension trading-only (use Pera or ARC-52 for custody) until Kraken ships a hardware-secure-element version.

Phase two (week 2-6): Foundry suite and Polygon-only deployment. Write the full Popperian Foundry suite first, get it to 100% green on a local anvil fork of Polygon, then deploy in order to Polygon mainnet: Senatus Safe, Aerarium (with Chainlog initialised), BonaToken (EIP-3009 enabled), Tabularium, Fides, Genius, SponsioPactum, Censura, Tessera, Curia. Wire openBDK L1Escrow on Ethereum mainnet against Unified Bridge

`0x2a3DD3EB832aF982ec71669E178424b10Dca2EDe` and ZkMinterBurner + NativeConverter on Polygon zkEVM mainnet. Verify every contract on Polygonscan via `forge verify-contract`. **Benchmark to pass:** a round-trip burn-and-mint of 100 BonaToken from L1 to L2 and back, with no balance discrepancy and Tabularium logs on both sides. **Stop condition:** if the L1→L2 bridge claim takes longer than one hour, audit the message-claim cron and the NeuralNode reference deployment at `0x024b464ec595F20040002237680026bf006e8F90` (DELTAVERSE NFT #1 on Polygon, per the `deltav-deltaverse/NeuralNode` GitHub repo) to understand its claim mechanics as a real-world reference. (GitHub)

Phase three (week 6–10): Algorand bridge, DAIO authorisation, Kraken trading live. Deploy the Algorand constitutional layer (BonaToken ASA, Fides + Tabularium PyTEAL apps, Wormhole NTT in locking mode on Algorand following the Algorand Foundation's NTT guide). Wire automindx to the thirteen Sun Tzu assessor agents with deterministic prompts and a published prompt hash. Connect the Curia executor to Parsec → Kraken extension and run the first ten trades at `clamp(volume * verdict_factor, min=$1, max=$100)` to validate the verdict-to-volume mapping. **Benchmark to pass:** ten consecutive trades whose realised execution matches the assessor verdict within $\pm 1\%$ slippage, Tabularium audit logs intact, and zero rate-limit violations against Kraken's tier-1 counter. **Stop condition:** if any single assessor's prompt drifts (detected via prompt-hash mismatch or a $>2\sigma$ score change from baseline) automatically demote that assessor's weight to zero in the next aggregation and emit a Censura event for human review.

Concrete actionable next steps for Gregory immediately:

1. **Pin dependencies.** `pip install "x402-avm[avm,fastapi,extensions]==<exact_version_from_pypi>"` after manually checking `pypi.org/project/x402-avm/#history`; do not use `pip install x402-avm` unpinned because the v2 → v2.6+ `algosdk` → `algokit-utils` migration broke the public API. Equivalently for npm: `npm install @x402-avm/avm@<pinned> @x402-avm/express@<pinned>`. Pin Foundry to a recent stable nightly (`foundryup --version stable`). (GitHub)
2. **Generate three new Validator keypairs offline** (one each on three different hardware devices — a Ledger Nano S Plus, a Trezor Safe 3, and an air-gapped Tails laptop), then create the Senatus 2-of-3 Gnosis Safe on Polygon mainnet at `https://app.safe.global` and record its address as the eventual owner of every BONAFIDE contract.
3. **Fork `agenticplace.pythai.net`'s `allchain.html`** into the mindx repo as a vendored snapshot, then write the `chain_registry.py` loader that parses it into the in-memory `ChainSpec` map. Validate against the known IDs: Algorand mainnet `algorand:wGHE2Pwvdvd7S12BL5Fa0P20EGYesN73ktiC1qzkkit8=`, Arc Testnet `eip155:5042002`, Polygon PoS `eip155:137`, Polygon zkEVM `eip155:1101`, Base `eip155:8453`, Ethereum `eip155:1`.
4. **Open three Kraken API keys** — one for `trading-pro` (full AddOrder/Cancel/Query/Balance permissions), one for `data-only` (public market data,

used by automindx for assessor inputs), one for `withdraw-only` (locked to one whitelisted Aerarium address and 24h delay), each with its own Redis nonce counter to avoid the documented multi-process nonce-conflict failure mode.

Caveats

Several pieces of the user-supplied spec do not map cleanly to canonical, in-production primitives and the architecture above silently corrected them in ways that are worth flagging.

The "1 Relayer + 3 Validator BFT topology" phrasing is not a named pattern in Agglayer, openBDK, Polygon CDK, or the Wormhole/NTT documentation. Agglayer's actual security model is ZK-proof-based (Unified Bridge + Pessimistic Proof + State Transition Proof), and the closest validator-style construct, Stake-the-Bridge, uses three administrative roles (Admin / Escrow manager / Risk manager), not a Byzantine-fault-tolerant quorum. The architecture substitutes a Senatus Gnosis Safe 2-of-3 for the EVM half and a Pera native-multisig 2-of-3 for the AVM half, which preserves the user's clearly-intended property (trust-minimisation through redundant cosigners) but should not be marketed using the literal phrase "BFT" because it is not a permissionless BFT consensus; it is a 2-of-3 multisig. If the user intended an actual BFT (HotStuff, Tendermint), that would require a dedicated chain or sidecar and is out of scope of this design.

The "openBDK" name does not resolve to a public, canonical project on GitHub or in Polygon's documentation. The deployment uses the `BuildOnPolygon/usdc-1x1y` pattern (the verified, in-production three-contract pattern of L1Escrow + ZkMinterBurner + NativeConverter against the Polygon Unified Bridge) as the canonical embodiment of what the user calls openBDK. If "openBDK" is the user's private branding of this pattern, all is well; if it refers to a different unreleased project, the architecture stands but the name should be reconciled.

The `parsec-wallet` GitHub organisation referenced by the user does not surface in public search as a project of clear scope; what does surface are unrelated namespaces (the CryptoNote-based `parsecnode/parsec` cryptocurrency wallet, `parallaxsecond/parsec` which is a CNCF hardware-security abstraction service, and the `parsec-finance` GitHub org). The design therefore treats Parsec strictly as the user's project name for the wallet abstraction layer described here, and the wallet-agnostic structural-typed protocol design is original to this document rather than wrapping an existing public `parsec-wallet` codebase. If a private `parsec-wallet` repo exists, the protocol contract above should be reconciled to it before code generation. `(GitHub + 2)`

Kraken Wallet's open-source repo (`krakenfx/wallet`) is real and MIT-licensed but as Kraken itself disclosed at launch, the secp256k1 signing on iOS happens inside the sandboxed app environment rather than in the Secure Enclave because Apple's CryptoKit does not expose

secp256k1. Any production deployment that wraps Kraken Wallet inherits that exposure surface; the Parsec extension architecture isolates it but cannot eliminate it. For high-value treasury operations, prefer the Pera Shared Account or Ledger-backed ARC-52 backends over the Kraken Wallet backend. [The Block](#)

The Kraken Pro Futures REST authentication is scheduled to change on 1 October 2025 (the old query-string-decoded Authn generation method is being phased out in favour of hashing the encoded query string). The deployment's Kraken Pro client must implement the new method; the deployment date is unambiguously in the future from the perspective of the protocol's original announcement but in the past from this report's reference date of 20 May 2026, so any pinned dependency must already use the post-1-Oct-2025 scheme. [Kraken](#)

The NeuralNode contract [0x024b464ec595F20040002237680026bf006e8F90](#) on Polygon is, per the public [deltav-deltaverse/NeuralNode](#) GitHub repo, the DELTAVERSE NFT #1 ("the iconic first DELTA VERSE concept"); the contract's role in the BONAFIDE architecture is therefore as a reference deployment to study (its vanity address [0x024b...2026...F90](#) was deliberately mined), not as a dependency or component. Direct Polygonscan verification was not retrievable from inside this research session, so the runbook should re-confirm the creator and deployment block before relying on it as a reference for L1Escrow message-claim mechanics. [GitHub](#)

Finally, the x402-avm versioning is in active flux: GoPlausible's documented breaking change "v2 → v2.6+: algosdk is no longer a dependency. The packages now use @algorandfoundation/algokit-utils@10.0.0-alpha.39 internally" means any code written against pre-v2.6 examples will not compile against current packages. The recommendation is to pin to the exact PyPI / npm version after manual confirmation at [pypi.org/project/x402-avm/#history](#) and [github.com/GoPlausible/x402-avm/releases](#), and to treat any dependency on [algosdk](#) as a regression that must be removed. [GitHub](#) [GitHub](#)